

MyVector

Sugeneruota Doxygen 1.13.2

skyrius 1

Klasės Indeksas

1.1 Klasės

Klasės, struktūros, sąjungos ir sąsajos su trumpais aprašymais:

`MyVector< T >` ??

skyrius 2

Failo Indeksas

2.1 Failai

Visų failų sąrašas su trumpais aprašymais:

include/ MyVector.h	??
Src/ MyVector.cpp	??

skyrius 3

Klasės Dokumentacija

3.1 MyVector< T > Klasė Šablonas

```
#include <MyVector.h>
```

Vieši Tipai

- using `value_type` = T
- using `size_type` = size_t
- using `reference` = T&
- using `const_reference` = const T&

Vieši Metodai

- `MyVector` ()
- `~MyVector` ()
- `MyVector` (const `MyVector` &other)
- `MyVector` & `operator=` (const `MyVector` &other)
- `MyVector` (`MyVector` &&other) noexcept
- `MyVector` & `operator=` (`MyVector` &&other) noexcept
- `MyVector` (std::initializer_list< T > init)
- void `push_back` (const T &element)
- void `pop_back` ()
- void `clear` ()
- bool `empty` () const
- void `resize` (size_t new_size)
- void `reserve` (size_t new_capacity)
- void `assign` (size_t n, const T &value)
- void `swap` (`MyVector` &other) noexcept
- size_t `get_size` () const
- size_t `get_capacity` () const
- T & `operator[]` (size_t index)
- const T & `operator[]` (size_t index) const
- T & `at` (size_t index)
- const T & `at` (size_t index) const
- T & `front` ()

- T & `back` ()
- void `shrink_to_fit` ()
- T * `begin` ()
- T * `end` ()
- const T * `begin` () const
- const T * `end` () const
- void `insert` (size_t pos, const T &value)
- void `erase` (size_t pos)

3.1.1 Tipo Aprašymo Dokumentacija

3.1.1.1 `const_reference`

```
template<typename T>  
using MyVector< T >::const_reference = const T&
```

3.1.1.2 `reference`

```
template<typename T>  
using MyVector< T >::reference = T&
```

3.1.1.3 `size_type`

```
template<typename T>  
using MyVector< T >::size_type = size_t
```

3.1.1.4 `value_type`

```
template<typename T>  
using MyVector< T >::value_type = T
```

3.1.2 Konstruktoriaus ir Destruktoriaus Dokumentacija

3.1.2.1 `MyVector()` [1/4]

```
template<typename T>  
MyVector< T >::MyVector ()
```

3.1.2.2 `~MyVector()`

```
template<typename T>  
MyVector< T >::~MyVector ()
```

3.1.2.3 MyVector() [2/4]

```
template<typename T>
MyVector< T >::MyVector (
    const MyVector< T > & other)
```

3.1.2.4 MyVector() [3/4]

```
template<typename T>
MyVector< T >::MyVector (
    MyVector< T > && other)    [noexcept]
```

3.1.2.5 MyVector() [4/4]

```
template<typename T>
MyVector< T >::MyVector (
    std::initializer_list< T > init)    [inline]
```

3.1.3 Metodų Dokumentacija

3.1.3.1 assign()

```
template<typename T>
void MyVector< T >::assign (
    size_t n,
    const T & value)
```

3.1.3.2 at() [1/2]

```
template<typename T>
T & MyVector< T >::at (
    size_t index)
```

3.1.3.3 at() [2/2]

```
template<typename T>
const T & MyVector< T >::at (
    size_t index) const
```

3.1.3.4 back()

```
template<typename T>
T & MyVector< T >::back ()
```

3.1.3.5 begin() [1/2]

```
template<typename T>
T * MyVector< T >::begin ()
```

3.1.3.6 begin() [2/2]

```
template<typename T>
const T * MyVector< T >::begin () const
```

3.1.3.7 clear()

```
template<typename T>
void MyVector< T >::clear ()
```

3.1.3.8 empty()

```
template<typename T>
bool MyVector< T >::empty () const
```

3.1.3.9 end() [1/2]

```
template<typename T>
T * MyVector< T >::end ()
```

3.1.3.10 end() [2/2]

```
template<typename T>
const T * MyVector< T >::end () const
```

3.1.3.11 erase()

```
template<typename T>
void MyVector< T >::erase (
    size_t pos) [inline]
```

3.1.3.12 front()

```
template<typename T>
T & MyVector< T >::front ()
```

3.1.3.13 get_capacity()

```
template<typename T>
size_t MyVector< T >::get_capacity () const
```

3.1.3.14 get_size()

```
template<typename T>
size_t MyVector< T >::get_size () const
```

3.1.3.15 insert()

```
template<typename T>
void MyVector< T >::insert (
    size_t pos,
    const T & value) [inline]
```

3.1.3.16 operator=() [1/2]

```
template<typename T>
MyVector< T > & MyVector< T >::operator= (
    const MyVector< T > & other)
```

3.1.3.17 operator=() [2/2]

```
template<typename T>
MyVector< T > & MyVector< T >::operator= (
    MyVector< T > && other) [noexcept]
```

3.1.3.18 operator[]() [1/2]

```
template<typename T>
T & MyVector< T >::operator[] (
    size_t index)
```

3.1.3.19 operator[]() [2/2]

```
template<typename T>
const T & MyVector< T >::operator[] (
    size_t index) const
```

3.1.3.20 pop_back()

```
template<typename T>
void MyVector< T >::pop_back ()
```

3.1.3.21 push_back()

```
template<typename T>
void MyVector< T >::push_back (
    const T & element)
```

3.1.3.22 reserve()

```
template<typename T>
void MyVector< T >::reserve (
    size_t new_capacity)
```

3.1.3.23 resize()

```
template<typename T>
void MyVector< T >::resize (
    size_t new_size)
```

3.1.3.24 shrink_to_fit()

```
template<typename T>
void MyVector< T >::shrink_to_fit ()
```

3.1.3.25 swap()

```
template<typename T>
void MyVector< T >::swap (
    MyVector< T > & other) [noexcept]
```

Dokumentacija šiai klasei sugeneruota iš šių failų:

- include/MyVector.h
- Src/MyVector.cpp

skyrius 4

Failo Dokumentacija

4.1 include/MyVector.h Failo Nuoroda

```
#include <stdexcept>
#include <initializer_list>
#include <algorithm>
#include <memory>
#include <string>
```

Klasės

- class `MyVector< T >`

4.2 MyVector.h

[Eiti į šio failo dokumentaciją.](#)

```
00001 #ifndef MYVECTOR_H
00002 #define MYVECTOR_H
00003
00004 #include <stdexcept>
00005 #include <initializer_list>
00006 #include <algorithm>
00007 #include <memory>
00008 #include <string>
00009
00010 template <typename T>
00011 class MyVector {
00012 private:
00013     T* arr;
00014     size_t sz;
00015     size_t cap;
00016
00017     void resize_internal();
00018
00019 public:
00020     using value_type = T;
00021     using size_type = size_t;
00022     using reference = T&;
00023     using const_reference = const T&;
00024
00025     MyVector();
00026     ~MyVector();
00027
00028     MyVector(const MyVector& other);
00029     MyVector& operator=(const MyVector& other);
```

```

00030
00031     MyVector(MyVector&& other) noexcept;
00032     MyVector& operator=(MyVector&& other) noexcept;
00033
00034     MyVector(std::initializer_list<T> init) {
00035         sz = init.size();
00036         cap = sz;
00037         arr = new T[cap];
00038         std::copy(init.begin(), init.end(), arr);
00039     }
00040
00041     void push_back(const T& element);
00042     void pop_back();
00043     void clear();
00044     bool empty() const;
00045
00046     void resize(size_t new_size);
00047     void reserve(size_t new_capacity);
00048     void assign(size_t n, const T& value);
00049     void swap(MyVector& other) noexcept;
00050
00051     size_t get_size() const;
00052     size_t get_capacity() const;
00053
00054     T& operator[](size_t index);
00055     const T& operator[](size_t index) const;
00056
00057     T& at(size_t index);
00058     const T& at(size_t index) const;
00059
00060     T& front();
00061     T& back();
00062
00063     void shrink_to_fit();
00064
00065     // Iterators
00066     T* begin();
00067     T* end();
00068     const T* begin() const;
00069     const T* end() const;
00070
00071     void insert(size_t pos, const T& value) {
00072         if (pos > sz) throw std::out_of_range("Insert position out of range");
00073         if (sz == cap) resize_internal();
00074         for (size_t i = sz; i > pos; --i) arr[i] = arr[i - 1];
00075         arr[pos] = value;
00076         ++sz;
00077     }
00078
00079     void erase(size_t pos) {
00080         if (pos >= sz) throw std::out_of_range("Erase position out of range");
00081         for (size_t i = pos; i < sz - 1; ++i) arr[i] = arr[i + 1];
00082         --sz;
00083     }
00084 };
00085
00086 // Čia realizacija iš karto po deklaracijos
00087
00088 template <typename T>
00089 MyVector<T>::MyVector() : sz(0), cap(1) {
00090     arr = new T[cap];
00091 }
00092
00093 template <typename T>
00094 MyVector<T>::~MyVector() {
00095     delete[] arr;
00096 }
00097
00098 template <typename T>
00099 MyVector<T>::MyVector(const MyVector& other) : sz(other.sz), cap(other.cap) {
00100     arr = new T[cap];
00101     std::copy(other.arr, other.arr + sz, arr);
00102 }
00103
00104 template <typename T>
00105 MyVector<T>& MyVector<T>::operator=(const MyVector& other) {
00106     if (this != &other) {
00107         delete[] arr;
00108         sz = other.sz;
00109         cap = other.cap;
00110         arr = new T[cap];
00111         std::copy(other.arr, other.arr + sz, arr);
00112     }
00113     return *this;
00114 }
00115
00116 template <typename T>

```

```

00117 MyVector<T>::MyVector(MyVector&& other) noexcept : arr(other.arr), sz(other.sz), cap(other.cap) {
00118     other.arr = nullptr;
00119     other.sz = 0;
00120     other.cap = 0;
00121 }
00122
00123 template <typename T>
00124 MyVector<T>& MyVector<T>::operator=(MyVector&& other) noexcept {
00125     if (this != &other) {
00126         delete[] arr;
00127         arr = other.arr;
00128         sz = other.sz;
00129         cap = other.cap;
00130         other.arr = nullptr;
00131         other.sz = 0;
00132         other.cap = 0;
00133     }
00134     return *this;
00135 }
00136
00137 template <typename T>
00138 void MyVector<T>::resize_internal() {
00139     cap = (cap == 0) ? 1 : cap * 2;
00140     T* new_arr = new T[cap];
00141     std::copy(arr, arr + sz, new_arr);
00142     delete[] arr;
00143     arr = new_arr;
00144 }
00145
00146 template <typename T>
00147 void MyVector<T>::push_back(const T& element) {
00148     if (sz == cap) {
00149         resize_internal();
00150     }
00151     arr[sz++] = element;
00152 }
00153
00154 template <typename T>
00155 void MyVector<T>::pop_back() {
00156     if (sz > 0) --sz;
00157 }
00158
00159 template <typename T>
00160 void MyVector<T>::clear() {
00161     sz = 0;
00162 }
00163
00164 template <typename T>
00165 bool MyVector<T>::empty() const {
00166     return sz == 0;
00167 }
00168
00169 template <typename T>
00170 void MyVector<T>::resize(size_t new_size) {
00171     if (new_size > cap) reserve(new_size);
00172     sz = new_size;
00173 }
00174
00175 template <typename T>
00176 void MyVector<T>::reserve(size_t new_capacity) {
00177     if (new_capacity > cap) {
00178         T* new_arr = new T[new_capacity];
00179         std::copy(arr, arr + sz, new_arr);
00180         delete[] arr;
00181         arr = new_arr;
00182         cap = new_capacity;
00183     }
00184 }
00185
00186 template <typename T>
00187 void MyVector<T>::assign(size_t n, const T& value) {
00188     if (n > cap) reserve(n);
00189     for (size_t i = 0; i < n; ++i) {
00190         arr[i] = value;
00191     }
00192     sz = n;
00193 }
00194
00195 template <typename T>
00196 void MyVector<T>::swap(MyVector& other) noexcept {
00197     std::swap(arr, other.arr);
00198     std::swap(sz, other.sz);
00199     std::swap(cap, other.cap);
00200 }
00201
00202 template <typename T>
00203 size_t MyVector<T>::get_size() const {

```

```

00204     return sz;
00205 }
00206
00207 template <typename T>
00208 size_t MyVector<T>::get_capacity() const {
00209     return cap;
00210 }
00211
00212 template <typename T>
00213 T& MyVector<T>::operator[](size_t index) {
00214     return arr[index];
00215 }
00216
00217 template <typename T>
00218 const T& MyVector<T>::operator[](size_t index) const {
00219     return arr[index];
00220 }
00221
00222 template <typename T>
00223 T& MyVector<T>::at(size_t index) {
00224     if (index >= sz) throw std::out_of_range("Index out of range");
00225     return arr[index];
00226 }
00227
00228 template <typename T>
00229 const T& MyVector<T>::at(size_t index) const {
00230     if (index >= sz) throw std::out_of_range("Index out of range");
00231     return arr[index];
00232 }
00233
00234 template <typename T>
00235 T& MyVector<T>::front() {
00236     if (sz == 0) throw std::out_of_range("Vector is empty");
00237     return arr[0];
00238 }
00239
00240 template <typename T>
00241 T& MyVector<T>::back() {
00242     if (sz == 0) throw std::out_of_range("Vector is empty");
00243     return arr[sz - 1];
00244 }
00245
00246 template <typename T>
00247 void MyVector<T>::shrink_to_fit() {
00248     if (sz < cap) {
00249         T* new_arr = new T[sz];
00250         std::copy(arr, arr + sz, new_arr);
00251         delete[] arr;
00252         arr = new_arr;
00253         cap = sz;
00254     }
00255 }
00256
00257 template <typename T>
00258 T* MyVector<T>::begin() {
00259     return arr;
00260 }
00261
00262 template <typename T>
00263 T* MyVector<T>::end() {
00264     return arr + sz;
00265 }
00266
00267 template <typename T>
00268 const T* MyVector<T>::begin() const {
00269     return arr;
00270 }
00271
00272 template <typename T>
00273 const T* MyVector<T>::end() const {
00274     return arr + sz;
00275 }
00276
00277 #endif // MYVECTOR_H

```

4.3 Src/MyVector.cpp Failo Nuoroda

```

#include "MyVector.h"
#include <string>

```